PRODUCT INVENTORY MANAGEMENT SYSTEM

Backend Project Documentation

Technologies: Node.js, Express.js, MongoDB, JWT Authentication

Tools: VS Code, Thunder Client, MongoDB Atlas

Table of Contents

1. 1. Introduction
2. 2. Project Overview
3. 3. Key Objectives
4. 4. Target Users
5. 5. Pre-Requisites
6. 6. Project Architecture
7. 7. Technical Architecture
8. 8. Database Design
9. 9. Features
10. 10. Roles and Responsibilities
11. 11. Project Setup
12. 12. Folder Structure
13. 13. API Endpoints
14. 14. API Testing
15. 15. Project Execution
16. 16.Screenshots or Demo
17. 17. Conclusion

1.Introduction

Project Title: Product Inventory Management

Team ID: NM2025TMID47842

Team Leader : A.DEEPAK & deepakdhanush439@gmail.com

Team Members:

T.K Dinesh & tkdineshtkdinesh3@gmail.com
P Deepak chakaravarthi & deepakdeepak10818@gmail.com
DINESH.G & dineshg862007@gmail.com

2. Project Overview

The Product Inventory Management System is a backend-focused web application developed using Node.js, Express.js, and MongoDB. The system provides secure RESTful APIs that allow users to manage their product inventory. It implements JWT-based authentication to ensure that only authorized users can access protected routes. Users can create, read, update, and delete products while maintaining secure ownership of their data.

3. Key Objectives

- Provide secure JWT-based authentication and authorization.
- Enable efficient product inventory management using REST APIs.
- Ensure each user can manage only their own products.
- Build scalable backend architecture using Node.js and Express.
- Provide reliable responses with validation and error handling.

4. Target Users

- Small business owners managing product stock.
- Freelancers tracking product inventory.
- Developers learning backend API development.
- Students working on backend or full-stack projects.

5. Pre-Requisites

Software	Purpose
Node.js	JavaScript runtime environment
MongoDB	Database for storing product data
npm	Dependency management
VS Code	Code editor for development

6. Project Architecture

The system follows a three-tier architecture consisting of API layer, business logic layer, and database layer. The Express server processes client requests, controllers handle business logic, and MongoDB stores application data.

7. Technical Architecture

- Controllers handle authentication and product operations.
- Models define database schemas using Mongoose.
- Routes define API endpoints.
- Middleware verifies JWT tokens for protected APIs.
- Environment configuration manages secrets and DB connections.

8. Database Design

Users Collection:

- _id
- name
- email
- password (hashed)
- timestamps

Products Collection:

- _id
- userId
- name
- price
- quantity
- description
- timestamps

9. Features

- User registration and login using JWT.
- Add new products to inventory.
- View products belonging to logged-in user.
- Update product details such as price or quantity.
- Delete products securely.
- Search products by name.

10. Roles and Responsibilities

- Register and login using secure credentials.
- Create new product entries.
- Modify product details when required.
- Delete products from inventory.
- Search products using keywords.

11. Project Setup

18. Create backend project folder.
19. Initialize Node.js project using npm init -y.
20. Install required dependencies.
21. Create environment variables in .env file.
22. Configure MongoDB connection.

12. Folder Structure

```
server/
│
├── src/
│   ├── controllers/
│   ├── db/
│   ├── middlewares/
│   ├── models/
│   ├── routes/
│   └── server.js
│
├── .env
├── package.json
```

13. API Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/register	Register new user
POST	/api/v1/auth/login	Login user
POST	/api/v1/products	Create product
GET	/api/v1/products	Get all products
PUT	/api/v1/products/:id	Update product
DELETE	/api/v1/products/:id	Delete product

14. API Testing

Thunder Client in VS Code was used to test all API endpoints including user authentication and product CRUD operations.

15. Project Execution

23. Open terminal in server folder.
24. Run npm install to install dependencies.
25. Add environment variables in .env.
26. Start the server using npm start or npm run dev.
27. Test APIs using Thunder Client.

16.Screenshots or Demo

Demo Video : Watch Here https://drive.google.com/file/d/1Ya3W9OoNe39d16-Zug7aXLQd7p7Oi9kJ/view?usp=sharing

Github link https://github.com/deepakdhanush439-oss/-Product-Inventory-Management.git

17. Conclusion

The Product Inventory Management System demonstrates backend development using Node.js, Express.js, and MongoDB. The project successfully implements user authentication, secure APIs, and product inventory management. This architecture can be extended to build full-stack applications.